COMPRENDRE LE PROJET

Observatoire crypto

Guide complet pour les entretiens techniques et le CV

Python - API - pandas - SQL - statistiques - visualisation - tests - GitHub



Collecte manuelle > contrôle qualité > upsert > analyse locale

Le dashboard lit SQLite. Les interactions ne déclenchent pas l'API.

Un fil conducteur : partir d'une réponse API, établir une observation fiable, puis expliquer exactement ce qu'un indicateur permet de conclure.

24 chapitres, questions-réponses, exercices corrigés et annexe des sources réellement livrées.

État vérifié : 22 cas de test locaux réussis. Démonstration sans clé API. Les capacités prévues et les limites sont distinguées des résultats démontrés.

Sommaire

Sommaire 2

1. Lire ce guide et situer le niveau de preuve 4

2. Le besoin et la présentation en une minute 5

3. Architecture et séparation des responsabilités 6

4. Installation, packaging et configuration 7

5. API CoinGecko et contrat de données 8

6. Résilience HTTP, délais et erreurs 9

7. Nettoyage : suivre une observation jusqu'à sa date 10

8. Bilan qualité et limites de mesure 11

9. Modèle SQL, upsert et idempotence 12

10. Exécutions, archives et échec partiel 13

11. Calendrier quotidien et calculs fondamentaux 14

12. Risque : volatilité et drawdown 15

13. Régression de tendance, pente et R^2 16

14. RSI, bandes, momentum et motifs 17

15. Dashboard, lecture et export 18

16. Démonstration, tests et niveau de confiance 19

17. Git, GitHub, CI et secrets 20

18. Exploitation, incidents et sécurité pratique 21

19. Formulations CV et présentation honnête du travail 22

20. Démonstration d'entretien en dix minutes 23

21. Questions d'entretien : réponses argumentées 24

22. Exercices avec résultats attendus 27

23. Feuille de route réaliste 28

24. Glossaire et sources pour continuer 29

Annexe A. Fiches des modules et lecture du code 30

A1. pyproject.toml 31

A2. .env.example 32

A3. .gitignore 33

A4. src/crypto_etl/config.py 34

A5. src/crypto_etl/client.py 35

A6. src/crypto_etl/transform.py 37

A7. src/crypto_etl/quality.py 38

A8. src/crypto_etl/storage.py 39

A9. src/crypto_etl/pipeline.py 41

A10. src/crypto_etl/indicators.py 42

A11. src/crypto_etl/trends.py 43

A12. src/crypto_etl/demo.py 45

A13. src/crypto_etl/cli.py 46

A14. dashboard/app.py 47

A15. tests/test_core.py 50

A16. tests/test_dashboard.py 53

A17. tests/test_trends.py 54

A18. .github/workflows/ci.yml 55

A19. .streamlit/config.toml 56

Annexe B. Commandes de révision 57

1. Lire ce guide et situer le niveau de preuve

Ce document sert à comprendre les décisions du projet, à montrer le code pertinent et à répondre aux objections. Il ne suffit pas de mémoriser des phrases : refaire les exemples, lancer les tests et modifier une fonction constitue une préparation plus solide. Une annexe reproduit le code source avec des numéros de ligne pour travailler sans connexion.

Le projet est un portfolio de traitement de données financières. Il met en œuvre une chaîne de données locale de bout en bout. Il ne constitue ni une plateforme de trading, ni un modèle prédictif, ni une expérience de production à grande échelle. La version livrée possède un client réel CoinGecko, mais aucune collecte réelle authentifiée n'a été validée dans cette session faute de clé API. Les tests utilisent des réponses simulées et la démonstration des prix synthétiques.

La dernière vérification locale a exécuté 22 cas de test avec succès. Ruff et son contrôle de formatage passent également. La matrice GitHub Actions est définie pour Windows/Linux et Python 3.10/3.12 ; sa présence ne prouve pas qu'une exécution distante a déjà réussi. Les 540 lignes de démonstration représentent 180 jours sur trois actifs, pas un volume de production ni une mesure de performance.

Parcours rapide avant entretien : lire les chapitres 2, 3, 8, 11, 15 et 19. Parcours technique : suivre les modules dans l'ordre config, client, transform, quality, storage, pipeline, indicators, trends, dashboard. Parcours de révision : exécuter les exercices et répondre aux questions sans regarder les propositions de réponse.

2. Le besoin et la présentation en une minute

Le besoin est de récupérer régulièrement un historique de prix, de contrôler sa qualité, de conserver une base cohérente et de comparer plusieurs actifs. Des appels API directement dans un graphique seraient insuffisants : ils mélangeraient réseau, calcul et affichage, consommeraient le quota à chaque filtre et rendraient les erreurs difficiles à reproduire.

Le périmètre initial comprend Bitcoin, Ethereum et Solana, identifiés par bitcoin, ethereum et solana. La devise est configurable, avec USD par défaut. Chaque collecte récupère les 180 derniers jours et conserve une observation par jour UTC terminé. L'utilisateur peut analyser un actif, comparer les performances et exporter sa sélection.

Proposition de pitch, à adapter après appropriation : « J'ai réalisé avec assistance IA un projet Python qui transforme des réponses CoinGecko en données quotidiennes analysables. Le pipeline contrôle les timestamps et les valeurs, archive les réponses et charge SQLite par upsert. Le dashboard lit uniquement la base locale. J'ai travaillé les cas difficiles : jours manquants, données révisées, limites HTTP et échecs partiels. Des tests hors réseau vérifient les calculs et la non-destruction des données. »

Pour une présentation de trois minutes, consacrer 30 secondes au besoin, 45 secondes au flux de données, 45 secondes à une difficulté réelle, 30 secondes aux tests et 30 secondes aux limites. La difficulté la plus démonstrative est la différence entre sept lignes et sept jours lorsque l'historique contient un trou.

3. Architecture et séparation des responsabilités



Collecte manuelle > contrôle qualité > upsert > analyse locale

Le dashboard lit SQLite. Les interactions ne déclenchent pas l'API.

Le flux logique est : commande CLI -> configuration -> client API -> archives JSON -> transformation/qualité -> SQLite -> indicateurs -> dashboard. Le dashboard ne déclenche pas le pipeline. L'actualisation des données et la consultation sont donc deux opérations indépendantes.

L'extraction récupère un objet JSON décodable. La transformation vérifie sa structure, convertit les séries et applique les règles métier. Le chargement écrit des lignes validées dans SQLite. Les indicateurs sont calculés à la lecture dans le dashboard, et ne sont pas stockés dans des colonnes SQL. Une correction de prix est ainsi répercutée au prochain calcul sans maintenance d'une table d'indicateurs.

L'archive brute et la base ne sont pas deux copies équivalentes : la première conserve la réponse décodable avant nettoyage, la seconde représente l'état courant validé par date. Une archive rend possible un rejeu et une enquête ; la base facilite les requêtes et l'unicité. Le système n'est pas un entrepôt de données historisant toutes les révisions.

config.py contient les paramètres ; client.py traite HTTP ; transform.py normalise ; quality.py résume les anomalies ; storage.py porte SQL ; pipeline.py coordonne ; cli.py expose les commandes ; demo.py fabrique des données ; indicators.py gère le calendrier ; trends.py ajoute les repères descriptifs ; dashboard/app.py présente les résultats.

Cette séparation permet d'injecter une fonction fetch dans le pipeline et une session simulée dans le client. Une fonction de calcul reçoit un tableau, pas une clé API ni un objet Streamlit. Le bénéfice concret est de tester les rendements à partir de trois prix sans serveur ni réseau.

4. Installation, packaging et configuration

Le fichier pyproject.toml décrit le paquet crypto-etl-local, les dépendances et le point d'entrée crypto-etl. Les sources se trouvent dans src/crypto_etl. L'installation éditable, python -m pip install -e ".[dev]", rend les modifications locales du paquet accessibles sans reconstruire manuellement une distribution à chaque changement.

Créer un environnement virtuel évite de mélanger les versions de ce projet avec celles d'autres logiciels. Sous PowerShell : python -m venv .venv, puis .venv\Scripts\Activate.ps1. Sous Linux : source .venv/bin/activate. Installer ensuite le paquet et lancer crypto-etl demo, puis streamlit run dashboard/app.py. Si PowerShell refuse l'activation, utiliser directement .venv\Scripts\python.exe et .venv\Scripts\streamlit.exe, sans modifier inutilement une politique globale.

Config est une dataclass figée. Ses attributs par défaut comprennent days=180, currency=usd, attempts=4, timeout=30 et l'URL Demo. frozen=True empêche de réassigner un champ après création ; ce n'est pas un mécanisme de chiffrement. Le champ api_key est exclu de repr pour limiter sa présence dans les affichages accidentels.

load_config lit COINGECKO_API_KEY, CRYPTO_CURRENCY, CRYPTO_DAYS et CRYPTO_DATA_DIR. La devise est mise en minuscules. Le nombre de jours doit être compris entre 1 et 365. La validation du code accepte une devise alphabétique, sans vérifier la liste réelle des devises CoinGecko : une devise inexistante sera refusée par l'API. Une chaîne non convertible en entier échoue dès le chargement.

Le fichier .env.example est un exemple documentaire. Il n'est pas chargé automatiquement : copier ce fichier en .env ne suffit donc pas. Dans PowerShell, utiliser $env:COINGECKO_API_KEY="..." ; dans un shell Linux, export COINGECKO_API_KEY='...'. Les chemins relatifs dépendent du dossier d'exécution. Un planificateur doit fixer son dossier de travail ou utiliser CRYPTO_DATA_DIR absolu.

Les versions sont bornées par des plages, et non verrouillées dans un lockfile. Cela facilite les mises à jour compatibles mais ne garantit pas une reproduction binaire exacte. Pour renforcer la reproductibilité, ajouter ultérieurement un verrouillage des versions et une stratégie de mise à jour contrôlée.

5. API CoinGecko et contrat de données

Le client appelle GET https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart. Les paramètres sont vs_currency, days et interval=daily. L'authentification Demo utilise l'en-tête x-cg-demo-api-key. Un GET ne signifie pas qu'il est gratuit en quota : chaque requête peut compter selon l'offre du fournisseur.

La réponse attendue contient prices, market_caps et total_volumes. Chaque série est une liste de paires [timestamp_en_millisecondes, valeur]. La présence de ces trois listes et la longueur de chaque paire sont contrôlées par validate_payload. Une liste vide est structurellement acceptable, puis conduit à une absence de données lors du contrôle métier.

Exemple pédagogique : prices=[[1704067200000,100],[1704153600000,105]], market_caps=[[1704067200000,1000]], total_volumes=[[1704153600000,80]]. Le prix du premier jour possède une capitalisation mais pas de volume. Celui du second possède un volume mais pas de capitalisation. Une jointure par position attribuerait des valeurs au mauvais jour.

La fonction fetch renvoie le JSON décodable ; la vérification de structure est exécutée par transform. Le pipeline archive donc aussi un objet JSON mal structuré avant de rejeter la série. Ce choix facilite le diagnostic. Un corps HTTP qui ne peut pas être décodé en JSON n'est pas conservé sous forme d'archive JSON dans cette version.

Un snapshot quotidien agrégé n'est pas une clôture officielle d'une place de marché. Les marchés crypto fonctionnent en continu et les données CoinGecko peuvent être nettoyées ou révisées. Le volume observé couvre les 24 heures associées au snapshot ; additionner plusieurs volumes glissants n'est pas équivalent à une quantité négociée sur une période disjointe.

L'offre Demo et ses limites doivent être vérifiées dans la documentation avant une utilisation réelle. Le projet retient une fenêtre compatible avec la limite de 365 jours documentée lors du développement. Il ne fixe pas un quota par minute comme vérité permanente : les offres commerciales peuvent évoluer.

6. Résilience HTTP, délais et erreurs

Client accepte une session HTTP et une fonction sleep injectables. En usage normal, requests.Session réutilise une session ; dans les tests, Mock remplace le réseau et l'attente. Le paramètre timeout=30 est transmis à chaque requête. Avec requests, ce timeout ne constitue pas une borne absolue sur toute la durée du pipeline : les tentatives et les attentes peuvent prolonger l'exécution.

Le code effectue quatre tentatives au maximum. Il réessaie après RequestException, HTTP 429 et les réponses serveur. Avec l'absence de Retry-After, les trois pauses précédant la quatrième tentative sont 1, 2 et 4 secondes. Le compteur attempts désigne le nombre total d'appels, pas le nombre de reprises supplémentaires.

Retry-After peut être un nombre de secondes ou une date HTTP. La fonction retry_delay transforme la valeur en attente positive ; si le parsing échoue, elle utilise le délai exponentiel. La version actuelle respecte une attente longue demandée par le serveur et n'introduit pas de jitter. Elle n'a pas non plus de deadline globale. Ces choix sont acceptables pour un petit outil manuel, mais doivent être discutés pour un service permanent.

HTTP 400, 401 et 403 entraînent un arrêt immédiat pour la crypto concernée. Répéter une clé invalide ne la rendra pas valide. Un JSON illisible échoue également immédiatement. Les autres cryptos continuent à être traitées, car le pipeline isole les erreurs par actif.

Les exceptions externes peuvent contenir des URLs ou des secrets. Le pipeline conserve seulement le type d'erreur et le nom de la crypto dans les logs. Cela réduit le risque de fuite mais rend le diagnostic moins précis. Une évolution serait de définir des exceptions métier avec code HTTP expurgé, tout en conservant l'absence de clé dans les messages.

7. Nettoyage : suivre une observation jusqu'à sa date

transform commence par valider la structure. Il détermine le jour UTC courant, puis une borne de départ égale à aujourd'hui moins days. Les tests peuvent fournir now pour rendre cette borne indépendante de l'horloge réelle. Une date sans fuseau fournie à now est interprétée comme UTC ; une date avec fuseau est convertie en UTC.

Chaque série devient un DataFrame contenant source_timestamp et sa valeur métier. Les timestamps sont convertis en nombres, les valeurs non convertibles sont retirées, puis les doublons par timestamp sont résolus en conservant la dernière occurrence de la série. La concaténation sur l'index des timestamps aligne les trois séries par clé exacte.

pd.to_datetime utilise unit='ms', utc=True et errors='coerce'. Un timestamp non représentable devient une date manquante. Après conversion, le fuseau est retiré pour stocker un jour normalisé sans ambiguïté : le champ s'appelle date_utc et sa convention reste UTC. Retirer le fuseau après conversion n'est pas la même chose que traiter une heure locale comme si elle était UTC.

Les valeurs sont converties en nombres. isfinite exclut NaN et les infinis. Prix strictement positif : sinon rejet de l'observation. Capitalisation et volume non négatifs : sinon valeur manquante conservée. Un volume égal à zéro est autorisé ; un prix égal à zéro ne l'est pas.

La fenêtre est fermée à gauche et ouverte à droite : start <= date < today. Le jour courant, encore incomplet, est exclu. Les lignes sont triées par timestamp source, puis la dernière ligne ayant un prix valide est retenue pour chaque jour. Si le dernier point du jour a un prix invalide, un point antérieur valide peut être retenu. Les autres séries ne sont pas automatiquement reportées depuis un autre timestamp.

L'absence de prix interdit de fabriquer une observation par interpolation. Ce choix évite de créer des rendements artificiellement lisses. Il distingue une information inconnue d'une information égale à zéro, ce qui aura des conséquences directes sur les fenêtres de calcul.

8. Bilan qualité et limites de mesure

quality_report construit la liste de toutes les dates attendues dans la fenêtre. La différence entre ce calendrier et les dates observées donne missing_days. La qualité ne se limite pas au nombre total de lignes : deux séries de 180 lignes peuvent couvrir des jours différents ou contenir des doublons.

Le bilan contient accepted, rejected, duplicate_timestamps, excluded_current_or_future, missing_days, missing_market_cap, missing_volume et large_changes. Une variation absolue strictement supérieure à 50 % est signalée sans supprimer le prix. Un événement exceptionnel n'est pas nécessairement une erreur de données.

Les compteurs ne forment pas une partition parfaite de la réponse brute. Les doublons sont comptés séparément dans chaque série et peuvent compter le même timestamp plusieurs fois. Les timestamps non numériques sont retirés avant le calcul de rejected. Les rejets sont évalués avant le filtre temporel ; certaines lignes rejetées peuvent donc être hors de la fenêtre utile. Il faut connaître ces nuances pour ne pas présenter le bilan comme un audit exhaustif de chaque élément JSON.

Le dashboard affiche les rapports de la dernière exécution, pas une agrégation historique complète. Des jours absents d'une collecte peuvent encore exister dans SQLite grâce à une collecte précédente. Le rapport de collecte et l'état global de la base répondent donc à deux questions différentes.

Améliorations possibles : compter séparément les timestamps invalides, les doublons intra-journaliers, les lignes hors fenêtre et les valeurs non finies ; définir un schéma de rapport versionné ; agréger la qualité de toute la base ; comparer les dates de fraîcheur par actif et par devise.

9. Modèle SQL, upsert et idempotence

daily_market_data contient coin_id, vs_currency, date_utc, source_timestamp, price, market_cap, volume_24h et collected_at. La clé primaire composite est (coin_id, vs_currency, date_utc). Une date seule n'est pas unique, puisque trois actifs et plusieurs devises peuvent partager le même jour.

source_timestamp conserve la précision du point retenu, tandis que date_utc identifie le jour métier. collected_at enregistre la collecte ayant fourni la version actuelle de la ligne. Les dates UTC sont stockées en texte ISO et les timestamps source en entier millisecondes. Les prix sont des REAL SQLite : adaptés à cette analyse descriptive, mais pas à une comptabilité monétaire exigeant un arrondi décimal exact.

INSERT ... ON CONFLICT ... DO UPDATE insère une ligne nouvelle ou remplace les attributs de la ligne existante. Si le prix d'un jour passe de 100 à 101 dans une collecte ultérieure, la base contient 101. Rejouer la même collecte ne crée pas de doublon. Les métadonnées collected_at changent toutefois : l'idempotence concerne l'état métier et l'unicité, pas l'identité bit à bit de chaque champ.

Le chargement d'une crypto est transactionnel : toutes ses lignes préparées sont exécutées dans le même bloc with db. Un échec SQL annule ce lot. La transaction ne couvre pas les trois actifs ensemble : une réussite Bitcoin est conservée même si Ethereum échoue ensuite. C'est un choix volontaire de disponibilité partielle.

Les valeurs sont liées avec des placeholders SQL, pas interpolées dans une chaîne. Cela évite les erreurs d'échappement et réduit les risques d'injection. Les contraintes CHECK contrôlent prix et valeurs non négatives, mais la validation Python reste importante. Le schéma ne possède pas de contrainte UNIQUE sur source_timestamp : plusieurs jours ne devraient pas partager un timestamp valide, mais la clé métier est la date.

SQLite convient au faible volume local et évite un serveur. La version actuelle ne configure ni WAL, ni stratégie élaborée de concurrence, ni migrations. Le context manager sqlite3 gère commit/rollback ; il ne ferme pas à lui seul explicitement la connexion. Le wrapper actuel ne comporte pas de finally db.close(), point à améliorer pour un usage long ou très fréquent.

10. Exécutions, archives et échec partiel

pipeline_runs enregistre run_id, started_at, ended_at, status, rows_processed et details. Un UUID hexadécimal sert de nom de dossier d'archive et d'identifiant de ligne. Le statut initial running est validé avant la collecte. La fin renseigne success, partial ou failed selon le nombre de cryptos chargées avec succès.

Le pipeline crée raw/<run_id>/<coin>.json pour les collectes réelles et raw-demo pour la démonstration. L'archive est écrite avant transformation, ce qui conserve la réponse pour comprendre un rejet. Il n'y a pas de transaction distribuée entre fichiers JSON et SQLite : un fichier peut être présent même si le chargement SQL échoue. Ce n'est pas un défaut caché ; c'est une propriété à documenter.

rows_processed mesure le nombre de lignes validées chargées, y compris les upserts de lignes déjà existantes. Ce n'est pas un compteur de nouvelles lignes insérées. Une exécution de 540 lignes traitées peut ne créer aucune nouvelle ligne si elle met seulement à jour des dates déjà présentes.

Si Ethereum échoue, ses dernières données valides restent en base. Le statut global devient partial si au moins une autre crypto a réussi. Aucun DELETE n'est exécuté. Une observation devenue invalide ou absente dans une nouvelle réponse ne supprime donc pas automatiquement une ligne antérieure ; seule une nouvelle ligne valide la remplace.

Une interruption brutale peut laisser un run running. Un disque plein peut empêcher l'archive ou le bilan final ; le code ne promet pas une journalisation parfaite malgré la panne du support de stockage. Pour aller plus loin, ajouter expiration des runs inachevés, erreurs métier expurgées, alertes et politique de rétention des archives.

11. Calendrier quotidien et calculs fondamentaux

indicators groupe d'abord par coin_id et vs_currency. Il trie les dates et applique asfreq('D') : un jour absent devient une ligne dont le prix est NaN. Les identifiants sont ensuite réattribués pour que les lignes de calendrier restent associées à la bonne série. Les calculs ne mélangent jamais Bitcoin et Ethereum ni USD et EUR.

Variation quotidienne : 100 * (P(t)/P(t-1)-1). Avec 100 puis 105, elle vaut 5 %. Avec 105 puis 100, elle vaut environ -4,7619 %, et non -5 %. Les rendements simples ne s'additionnent pas exactement : deux hausses de 10 % produisent 21 % de performance, car 1,1 * 1,1 = 1,21.

Variation hebdomadaire : 100 * (P(t)/P(t-7)-1). Sur un calendrier quotidien, shift(7) pointe vers la date exacte sept jours avant. Si le prix de cette date manque, le résultat reste NaN. Un trou intermédiaire n'invalide pas nécessairement ce rendement de point à point si les deux prix extrêmes existent ; il invalide en revanche les fenêtres exigeant tous les points.

Moyenne mobile n jours : somme des n prix divisée par n, avec min_periods=n. Les prix 100,101,102,103,104,105,106 donnent une MM7 de 103 au septième jour. Les six premiers jours restent indisponibles. Une seule valeur manquante dans une fenêtre empêche sa moyenne complète.

La moyenne mobile est calculée avant le filtre d'affichage. Si l'utilisateur sélectionne une période commençant en juin et que mai est disponible, la MM30 du début juin peut utiliser les derniers jours de mai. Filtrer d'abord créerait artificiellement 29 résultats indisponibles au début de chaque sélection.

La performance de sélection et la base 100 sont calculées après le filtre. Le premier prix disponible devient 100 ; un dernier prix de 120 comparé à un premier de 100 donne une base finale 120 et une performance de 20 %. Si un actif commence plus tard faute de données, il possède une date de référence différente : la comparaison n'est alors pas parfaitement synchronisée.

12. Risque : volatilité et drawdown

La volatilité sur 30 jours est l'écart-type des 30 derniers rendements quotidiens exprimés en pourcentage. L'écart-type échantillonnal utilise ddof=1 : somme des écarts au rendement moyen au carré, divisée par n-1, puis racine carrée. Il nécessite donc 31 prix consécutifs pour produire 30 rendements valides.

Exemple court pour comprendre ddof : pour les rendements 1 %, 2 %, 3 %, la moyenne est 2 %. Les écarts sont -1,0,1 point de pourcentage ; leur somme des carrés est 2. En divisant par 3-1, puis en prenant la racine, l'écart-type vaut 1 point de pourcentage. Avec ddof=0, il serait environ 0,8165.

La volatilité de ce projet n'est pas annualisée. Multiplier par racine(365) serait une convention supplémentaire avec des hypothèses sur les rendements ; ce n'est pas appliqué ici. Volatilité élevée ne signifie pas automatiquement baisse : une série en forte hausse peut aussi être volatile. Faible volatilité ne prouve pas l'absence de risque extrême.

Le drawdown vaut 100 * (prix / maximum cumulé - 1). Pour 100,120,90, les drawdowns sont 0 %,0 %,-25 %. Pour revenir de 90 à 120 après cette perte, il faut une hausse de 33,333 %, et non de 25 %. La valeur présentée à la fin est le drawdown courant ; ce n'est pas le maximum drawdown de toute la période.

Le maximum est recalculé sur la période sélectionnée. Un sommet avant le début choisi n'est pas utilisé. Sur des dates de prix manquant, le résultat est manquant ; lorsque les prix reprennent, le maximum cumulé des observations disponibles reste la référence. Le projet n'estime pas de VaR, de CVaR, de bêta ni de risque de portefeuille.

13. Régression de tendance, pente et R²

Sur 30 prix quotidiens consécutifs, trends.py ajuste une droite prix = a + b*x où x vaut 0,1,...,29. numpy.polyfit calcule les coefficients au moindre carré. La pente b représente une variation de prix par jour dans la devise de la série.

Pour comparer les actifs, la pente est divisée par le prix moyen de la fenêtre puis multipliée par 100. Une pente de 1 USD/jour sur un prix moyen de 144,5 USD correspond à environ 0,692 % du prix moyen par jour. Ce n'est ni un rendement composé quotidien ni une prévision : c'est une pente normalisée de l'ajustement historique.

$R^2 = 1 - SSE/SST$, où SSE est la somme des résidus au carré et SST la somme des écarts au prix moyen au carré. Une droite parfaite non constante produit $R^2=1$. Un prix constant produit SST=0 ; le code retourne conventionnellement $R^2=0$ afin d'éviter une division par zéro. Un R^2 élevé ne signifie pas que la prochaine observation suivra la droite.

La classe haussière exige une pente normalisée strictement supérieure à 0,1 %/jour et R^2 au moins égal à 0,5. La classe baissière exige une pente strictement inférieure à -0,1 et le même R^2 minimal. Sinon le libellé est « Sans tendance linéaire nette ». Moins de 30 prix consécutifs donne « Indisponible ».

Ces seuils sont des choix explicites de présentation, sans optimisation ni validation statistique de rentabilité. « Sans tendance linéaire nette » ne prouve pas l'absence de cycles, de rupture ou de structure non linéaire. Une évolution possible serait une régression sur logarithmes, une analyse de sensibilité des seuils et une validation hors échantillon, chacune clairement distinguée du code actuel.

14. RSI, bandes, momentum et motifs

Le RSI 14 utilise 14 variations quotidiennes : gain = max(diff,0), perte = max(-diff,0), puis leurs moyennes arithmétiques. RSI = 100 - 100/(1 + gain_moyen/perte_moyenne). Il faut 15 prix consécutifs. Cette implémentation utilise des moyennes simples ; elle ne reproduit pas le lissage récursif du RSI de Wilder fréquemment affiché ailleurs.

Si toutes les variations sont positives, le RSI vaut 100 ; si elles sont négatives, 0 ; si elles sont toutes nulles, 50. Avec un gain moyen de 2 et une perte moyenne de 1, le ratio vaut 2 et le RSI environ 66,667. Les lignes 30 et 70 servent de repères visuels et ne déclenchent aucun achat ni vente. Un RSI élevé peut rester élevé pendant une tendance longue.

Les bandes de Bollinger utilisent moyenne 20 jours plus ou moins deux écarts-types échantillonnaux des prix. Il s'agit de dispersion des niveaux de prix, contrairement à la volatilité calculée sur les rendements. Sortir d'une bande n'implique pas un retour certain à la moyenne. Les bandes ne sont pas des intervalles de confiance garantis à 95 %.

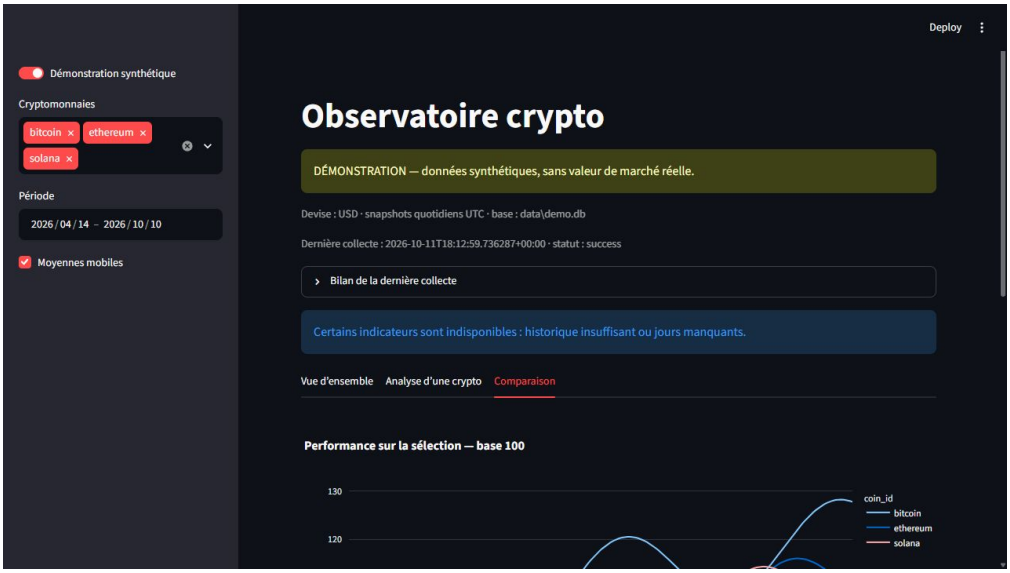
Le momentum 30 jours est un rendement de point à point entre les prix des dates t et t-30. Il exige les deux extrémités ; les trous intermédiaires ne le rendent pas nécessairement nul. Une pente de régression et ce momentum peuvent diverger : la première utilise tous les prix d'une fenêtre et le second seulement deux prix.

Le canal est le minimum et le maximum des 20 prix quotidiens précédents, calculés avec shift(1), puis rolling(20). Exclure le jour courant est essentiel : sinon le prix ne pourrait pas dépasser strictement un maximum qui l'inclut. Un nouveau sommet signifie prix > maximum précédent ; l'égalité n'est pas une cassure. Un nouveau creux suit la règle symétrique.

Un croisement haussier est détecté lorsque MM7-MM30 passe de <=0 à >0. Un croisement baissier passe de >=0 à <0. C'est un événement ponctuel : le jour suivant un croisement, rester au-dessus ne produit pas un nouveau croisement. Le calcul exige les moyennes actuelles et celles du jour précédent.

Ces motifs sont causaux : ils utilisent seulement des données présentes ou passées. Le test de préfixe vérifie que calculer 45 jours seuls donne les mêmes résultats que prendre les 45 premiers jours d'un calcul de 60 jours. Cela ne transforme pas les motifs en stratégie validée. Aucun double sommet, tête-épaules ou chandelier OHLC n'est reconnu dans cette version.

15. Dashboard, lecture et export



Capture de la démonstration : comparaison de séries synthétiques. La capture précède les ajouts de tendance.

Streamlit exécute le script lors des interactions. Il lit SQLite, calcule les indicateurs, filtre la période et construit les graphiques Plotly. La version actuelle n'utilise pas de cache Streamlit : c'est simple et acceptable pour quelques centaines ou milliers de lignes, mais les lectures et calculs sont répétés. Aucune interaction n'appelle CoinGecko.

Le mode synthétique est activé par défaut, avec une bannière explicite. La base demo.db est distincte de market.db et les exports portent crypto-demo.csv ou crypto-coingecko.csv. Le choix d'une devise différente nécessite de collecter ou générer des lignes correspondant à cette devise ; le dashboard ne fait pas une conversion de change à la volée.

La vue d'ensemble présente les dernières observations disponibles dans la sélection, leurs dates, variations et performances. La vue d'analyse affiche une synthèse de tendance, prix et moyennes, bandes/canal, RSI, événements, volume, volatilité et drawdown. La comparaison trace les prix normalisés et un tableau par actif.

Les valeurs manquantes doivent rester visibles ou provoquer des interruptions de courbe, sans connexion artificielle de points éloignés. Un message indique les fenêtres insuffisantes. Il peut apparaître sur une période longue pourtant complète parce que ses premiers jours n'ont pas encore 30 rendements historiques. Il faut lire ce message comme un état de certains points, pas comme l'échec de toute la série.

L'export porte sur le tableau sélectionné, avec les indicateurs calculés et les éventuelles lignes calendaires manquantes. Il est destiné à l'analyse dans un autre outil. Il ne contient pas de clé API. Un filtre sur une crypto n'efface pas ses voisines de SQLite ; il ne fait que limiter le tableau en mémoire.

La capture historique du README montre la démonstration, pas des prix réels. L'interface utilise encore certains paramètres Streamlit dépréciés, notamment use_container_width. Ils fonctionnent dans l'environnement vérifié mais justifient une future mise à jour API avant une suppression par le fournisseur.

16. Démonstration, tests et niveau de confiance

demo.py crée une série déterministe combinant progression et sinusoïde. Des prix de base différents servent à illustrer l'intérêt de la normalisation. La date de fin suit le dernier jour UTC terminé, donc deux exécutions à des dates différentes ne produisent pas le même calendrier. Aucun appel HTTP ni secret n'est nécessaire.

tests/test_core.py vérifie quatre réponses mal formées, une réponse vide, l'alignement, les doublons, le dernier point valide du jour, les valeurs invalides, les données du jour, les calculs à la main, la révision et les reprises HTTP. Les tests paramétrés comptent plusieurs cas : 22 cas exécutés ne signifie pas 22 fonctions distinctes.

Le test d'upsert charge deux fois le même tableau, modifie un prix puis recharge. Il vérifie trois lignes finales et la nouvelle valeur. Le test d'échec partiel effectue une première collecte valide puis fait échouer Ethereum ; il vérifie que son ancien prix reste en place et que Bitcoin est actualisé.

tests/test_trends.py couvre séries croissantes et décroissantes, série constante, bandes, RSI extrêmes, trous, causalité et caractère ponctuel du croisement. Les exemples contrôlés sont indispensables : un graphique visuellement plausible peut masquer un décalage de sept dates ou un calcul sur la mauvaise unité.

tests/test_dashboard.py utilise AppTest de Streamlit. Il vérifie l'état vide, les trois onglets, l'absence d'exception et les filtres d'actifs. Ce n'est pas un test exhaustif de tous les navigateurs ni de l'esthétique. Même sans réseau extérieur, Streamlit utilise une boucle asyncio et des sockets locales ; elles étaient bloquées dans le sandbox de la session et le test a été exécuté hors de ce sandbox.

Des tests supplémentaires seraient utiles sur Retry-After en date HTTP, erreurs 5xx, jours multi-devises, rollback SQL et incidents de stockage. La couverture existante réduit des risques précis ; elle ne permet pas d'affirmer zéro bug, ni un pourcentage de couverture non mesuré.

17. Git, GitHub, CI et secrets

Git versionne le projet local ; GitHub héberge un dépôt distant. Un commit local ne publie rien. git add prépare le contenu du prochain commit ; git commit crée un jalon ; git status montre l'écart du dossier de travail ; git log affiche les jalons ; git push transmet des commits au dépôt distant configuré.

L'historique du projet comporte l'initialisation, le pipeline, le dashboard et les tests, la validation, l'ajout des tendances et la correction de surveillance Streamlit. Les commits ont reçu une identité explicitement passée via git -c, sans modifier la configuration globale. Une adresse d'auteur figure dans les commits Git : un dépôt public expose normalement ces métadonnées.

.gitignore exclut .env, .venv, data, bases SQLite, logs et caches. Il n'efface pas un secret déjà commité : dans ce cas, il faut révoquer le secret et nettoyer l'historique avec une procédure adaptée. Un scan de motifs courants réduit le risque sans garantir mathématiquement l'absence de tout secret arbitraire.

Le workflow GitHub Actions s'exécute sur push et pull_request. Sa matrice prévoit quatre combinaisons : deux systèmes et deux versions Python. Chaque job installe le projet, exécute Ruff, vérifie le formatage et lance pytest. Il n'utilise aucune clé CoinGecko et ne collecte pas de données.

Une CI verte signifie que les vérifications définies passent dans les environnements exécutés. Elle ne prouve pas l'exactitude économique d'une stratégie, ni la disponibilité de l'API réelle, ni le bon fonctionnement de toutes les versions futures. Une collecte planifiée GitHub nécessiterait une gestion de stockage persistant distincte ; elle n'est pas incluse.

18. Exploitation, incidents et sécurité pratique

La commande crypto-etl update renvoie 0 si toutes les cryptos réussissent, 1 si le pipeline est partiel ou échoué ; argparse peut renvoyer 2 en cas d'usage ou de clé manquante. Un ordonnanceur doit traiter ces codes au lieu de se fier seulement à un fichier qui existe.

Sous Windows, configurer le Planificateur de tâches avec le chemin absolu du Python du venv, les arguments -m crypto_etl.cli update et un dossier de démarrage stable. Sous Linux, un wrapper privé peut charger la clé depuis un fichier protégé puis être lancé par cron et flock. Choisir une heure après la disponibilité du snapshot et préciser le fuseau de l'ordonnanceur.

Éviter deux collectes simultanées. SQLite peut refuser une écriture concurrente. Sauvegarder une base avec l'API Connection.backup plutôt que copier à l'aveugle un fichier en cours d'écriture. Prévoir rotation des logs et rétention des archives. Ces procédures sont documentées ; aucune tâche quotidienne n'a été créée automatiquement pendant le développement.

Incident réel : KeyError 'trend'. L'interface mise à jour attendait une colonne produite par le nouveau module, alors que le serveur gardait l'ancien module en mémoire. Un processus neuf produisait la colonne. Le correctif a été de redémarrer Streamlit et de surveiller src via .streamlit/config.toml. Cette configuration aide le rechargement ; elle ne remplace pas un redémarrage contrôlé lors d'un déploiement.

Incident d'environnement : l'installation pip échouait sur la chaîne de certificats. La solution a transmis les autorités de certification Windows à pip, y compris ses sous-processus, au lieu de désactiver la vérification TLS. Ce point distingue la résolution d'un problème local de l'introduction d'une faiblesse de sécurité.

19. Formulations CV et présentation honnête du travail

Titre possible : « Projet personnel - pipeline ETL crypto et dashboard analytique Python ». Ligne synthétique : « Mise en œuvre d'une chaîne CoinGecko -> pandas -> SQLite -> Streamlit/Plotly, avec chargements idempotents, contrôles qualité et 22 cas de test hors réseau. »

Pour un profil data engineer : « Structuration d'un pipeline local avec archives brutes, normalisation UTC, clé composite SQL, upserts transactionnels et suivi des échecs partiels ; configuration d'une CI Windows/Linux. » Pour un profil data analyst : « Calcul et visualisation de rendements, volatilité, drawdown et tendances descriptives ; comparaison base 100 et gestion explicite des données manquantes. »

Ajouter seulement ce que vous êtes capable d'expliquer. Ne pas écrire « prédiction du marché », « plateforme haute disponibilité », « millions de lignes traitées », « stratégie rentable » ou « production cloud » : ces éléments ne sont pas réalisés. Dire « CI configurée » si son exécution distante n'est pas encore vérifiée ; dire « client API implémenté » plutôt que « collecte réelle validée » tant qu'une clé n'a pas été testée.

L'assistance IA a servi à produire du code, des tests et la documentation. Vous pouvez dire : « J'ai utilisé un assistant pour accélérer l'implémentation, puis j'ai étudié les décisions, exécuté les tests et vérifié les cas limites. Je peux montrer les fonctions et expliquer leur comportement. » Ne présentez pas comme personnellement maîtrisée une partie que vous n'avez pas encore étudiée.

Pour montrer votre appropriation, préparez une modification simple en direct : ajouter une devise de test, changer le seuil de forte variation ou écrire un test sur un jour manquant. Expliquer pourquoi la modification est sûre a plus de valeur que réciter une liste de bibliothèques.

20. Démonstration d'entretien en dix minutes

Minute 0-1 : expliquer le besoin, le périmètre et l'état synthétique des données. Minute 1-2 : montrer le schéma du flux et l'indépendance du dashboard. Minute 2-3 : afficher la vue d'ensemble et comparer les bases 100. Minute 3-4 : montrer tendance, RSI et canal en distinguant observation et prévision.

Minute 4-5 : ouvrir transform.py et expliquer la jointure par timestamp. Minute 5-6 : ouvrir storage.py et montrer la clé composite ainsi que ON CONFLICT. Minute 6-7 : montrer le test de révision historique et celui d'échec partiel. Minute 7-8 : lancer pytest et Ruff dans l'environnement préparé.

Minute 8-9 : montrer une limite réelle, par exemple les fenêtres incomplètes ou l'absence de lockfile. Minute 9-10 : proposer l'amélioration suivante avec un critère de réussite : collecte réelle expurgée, fermeture explicite des connexions, tests supplémentaires, puis stockage plus robuste si le volume justifie cette évolution.

Avant la réunion : générer la démo, lancer le serveur, ouvrir les fichiers et vérifier les commandes. Prévoir le PDF et une capture si le navigateur ou le réseau échoue. Ne pas afficher une clé en partage d'écran, ne pas dépendre d'un appel CoinGecko pour démontrer les transformations, et ne pas masquer les données synthétiques.

21. Questions d'entretien : réponses argumentées

Pourquoi SQLite plutôt que PostgreSQL ?

Pour un outil local mono-utilisateur et quelques centaines de points par collecte, SQLite réduit l'installation et permet de démontrer SQL, contraintes et transactions. PostgreSQL devient intéressant si plusieurs services écrivent simultanément, si les accès doivent être administrés ou si le volume et les requêtes dépassent ce cadre. Le choix dépend des besoins mesurés, pas du prestige d'un outil.

Pourquoi ne pas tout garder en CSV ?

Un CSV est simple pour l'échange, mais ne garantit pas lui-même une clé unique ni un upsert transactionnel. SQLite fournit des contraintes et des requêtes paramétrées. Le CSV reste utile comme export de sélection, tandis que JSON conserve les réponses avant transformation.

Quelle est la différence entre ETL et ELT ici ?

Les données sont nettoyées en Python avant le chargement dans la table d'analyse : le chemin principal est ETL. L'archive brute est néanmoins conservée. Dans un ELT, on chargerait généralement les données brutes dans un stockage analytique avant transformation ; le fait d'avoir une archive de fichiers ne transforme pas automatiquement cette application en architecture ELT complète.

Comment garantissez-vous l'absence de doublons ?

La clé primaire composite est la garantie finale. La transformation résout aussi les timestamps dupliqués et les multiples observations journalières. L'upsert traite le conflit sans ajouter une seconde ligne. Les tests chargent deux fois la même série et contrôlent le nombre de lignes.

Si l'API corrige un prix vieux de dix jours ?

La fenêtre de 180 jours est recollectée. La ligne corrigée remplace la ligne existante par upsert. Les indicateurs sont recalculés au prochain affichage, donc leurs fenêtres sont actualisées. La base n'historise pas les anciennes versions de cette ligne ; les archives de collecte peuvent aider à les retrouver.

Pourquoi sept lignes ne suffisent pas pour sept jours ?

Une série peut manquer un jour. Un décalage de sept lignes saute alors huit jours calendaires. Reconstituer un index quotidien avec asfreq('D') fait du shift(7) un décalage de sept dates. Si une extrémité manque, le rendement reste indisponible au lieu d'utiliser une autre date.

Pourquoi ne pas remplir les prix manquants ?

Un remplissage peut créer des rendements nuls artificiels et réduire la volatilité estimée. L'application préfère conserver l'incertitude. Une méthode d'imputation pourrait être pertinente pour un autre usage, mais elle devrait être explicite, évaluée et séparée des données observées.

Le prix du jour est-il vraiment une clôture ?

Non. C'est un snapshot CoinGecko, agrégé et daté en UTC. La règle du projet retient le dernier point valide du jour terminé. L'interface ne l'appelle pas clôture officielle, et les figures de chandeliers ne sont pas calculées à partir de ces seules données.

Que signifie 22 tests réussis ?

Cela signifie 22 cas exécutés localement, y compris des paramètres de tests. Ils couvrent des risques définis, pas toutes les entrées possibles. Je peux citer les cas d'alignement, d'upsert, d'erreur partielle et de causalité. Je ne revendique pas un taux de couverture qui n'a pas été mesuré.

Les indicateurs donnent-ils un signal d'achat ?

Ils donnent des repères descriptifs. Aucun ordre n'est envoyé, aucune performance de stratégie n'est évaluée, et les seuils ne sont pas optimisés. Avant toute revendication prédictive, il faudrait un protocole hors échantillon avec coûts, liquidité, biais et comparaison à une référence.

Comment évitez-vous d'utiliser le futur ?

Les fenêtres de calcul se terminent au jour observé, le canal précédent décale le prix d'un jour et aucun rolling centré n'est utilisé. Un test compare les résultats sur un préfixe seul à ceux du même préfixe extrait d'une série plus longue. Les filtrages de période pour base 100 sont des transformations d'affichage, pas des signaux de trading testés.

Comment gérez-vous les secrets ?

La clé provient d'une variable d'environnement et passe dans un en-tête. Elle est exclue de repr, des logs et des fichiers versionnés. Les messages d'exception externes ne sont pas recopiés. Ces mesures ne chiffrent pas l'environnement ; elles réduisent l'exposition accidentelle dans le cadre local.

Que feriez-vous à dix millions de lignes ?

Je mesurerais d'abord les lectures, le temps de calcul et la concurrence. Ensuite : collecte incrémentale avec fenêtre de recouvrement, ingestion par lots, index adaptés, PostgreSQL ou stockage colonnaire selon les accès, cache invalidé sur version de données et calculs préagrégés. Je ne migrerais pas sans besoin mesuré.

Pourquoi la CI n'actualise-t-elle pas les prix ?

Les tests doivent rester déterministes et sans secret. Une collecte planifiée aurait un autre objectif et exigerait des identifiants, du stockage persistant et des alertes. Séparer les deux évite de confondre qualité du code et disponibilité de la source externe.

Quel bug réel avez-vous rencontré ?

L'interface demandait trend, absent du module encore chargé en mémoire par Streamlit. La colonne existait dans un processus neuf. Le diagnostic a distingué données et cycle de chargement du code ; le correctif a redémarré le serveur puis ajouté la surveillance de src. Le test neuf seul n'aurait pas révélé cet état d'un serveur ancien.

Quel compromis changeriez-vous en premier ?

Je fermerais explicitement les connexions SQLite, enrichirais les erreurs métier expurgées et validerais une collecte réelle. Puis j'ajouterais les tests manquants sur Retry-After et rollback. Ces améliorations renforcent la fiabilité avant d'ajouter davantage de graphiques.

22. Exercices avec résultats attendus

Exercice 1 : prix 100 puis 105 puis 110. Le premier rendement est indisponible, le deuxième vaut 5 %, le troisième environ 4,7619 %. La performance totale vaut 10 %, pas 9,7619 %. Vérifier en multipliant les facteurs de rendement.

Exercice 2 : prix 100,120,90. Base 100 : 100,120,90. Drawdown : 0,0,-25 %. Performance finale : -10 %. Expliquer pourquoi le drawdown final et la performance finale sont différents : ils n'ont pas le même prix de référence.

Exercice 3 : supprimer le quatrième jour d'une série de 40 prix. Attendre une ligne de calendrier manquante, des rendements quotidiens indisponibles autour du trou et des moyennes complètes interrompues. Le rendement à sept jours manque seulement si l'une de ses deux extrémités manque.

Exercice 4 : série constante à 100 pendant 40 jours. Attendre RSI=50 après initialisation, bandes égales à 100, volatilité=0 après 30 rendements, R^2 conventionnel=0 et absence de cassure. Les résultats initiaux restent manquants malgré la simplicité de la série.

Exercice 5 : série croissante de 100 à 159 par pas de 1. Sur la dernière fenêtre de 30 jours, le prix moyen est 144,5, la pente brute 1, la pente normalisée environ 0,692 %, R^2=1, RSI=100 et résistance précédente=158. Le dernier prix 159 dépasse la résistance et donne un nouveau sommet 20 jours.

Exercice 6 : charger deux fois trois lignes, puis corriger la première. Attendre trois lignes en base, avec le prix corrigé. Comparer aussi collected_at : il est actualisé. Rédiger un test qui vérifie le résultat métier, pas uniquement le retour de la fonction.

Exercice 7 : simuler 429 puis 200 avec Retry-After=3. Attendre deux appels et une seule pause de trois secondes. Simuler quatre réponses 429 : attendre quatre appels, trois pauses et un échec. Simuler 401 : attendre un appel sans pause.

Exercice 8 : rendre une crypto indisponible après une collecte réussie. Attendre partial, conservation de son ancien prix et actualisation des autres actifs. Expliquer ce qui se passe si les trois échouent et ce que montre la fraîcheur des observations.

23. Feuille de route réaliste

Priorité 1, fiabilité : valider le client sur une vraie clé, ajouter fermeture explicite des connexions, deadline et erreurs métier expurgées, tester dates Retry-After, serveur 5xx et rollback. Critère : résultats attendus documentés, tests reproductibles, aucun secret dans les sorties.

Priorité 2, audit : préciser les compteurs qualité, stocker une version du contrat de transformation, gérer les exécutions abandonnées, organiser la rétention des archives et la sauvegarde. Critère : relier une ligne courante à une collecte et expliquer chaque rejet de façon non ambiguë.

Priorité 3, expérience : libellés français et unités homogènes dans les tableaux, cache invalidé sur la dernière collecte, comparaison sur dates communes optionnelle, renouvellement des paramètres Streamlit dépréciés. Critère : chargement mesuré et absence de valeurs périmées après mise à jour.

Priorité 4, analyse : corrélations de rendements sur jours communs, maximum drawdown, rendements logarithmiques et diagnostics de sensibilité. Pour une étude prédictive, créer un projet séparé avec protocole de validation, référence naïve et coûts de transaction. Ne pas faire passer un nouvel indicateur descriptif pour une preuve de prévision.

Priorité 5, industrialisation selon besoin : incrémental avec recouvrement, scheduler surveillé, packaging verrouillé, migrations, base serveur et déploiement authentifié. Docker reste une option future, pas une exigence satisfaite par ce dépôt actuel.

24. Glossaire et sources pour continuer

API : contrat d'échange entre logiciels. Endpoint : URL d'une ressource API. JSON : format de données textuelles. Timestamp : instant numérique référencé à une origine. UTC : référence temporelle commune. Snapshot : observation à un instant. ETL : extraire, transformer, charger.

DataFrame : tableau pandas avec index et colonnes. NaN : valeur numérique manquante en mémoire. NULL : valeur absente en SQL. Upsert : insertion ou mise à jour sur conflit. Transaction : groupe d'écritures validé ou annulé. Idempotence : répéter une opération conserve le même état métier visé.

Rendement : variation relative d'un prix. Volatilité : dispersion des rendements selon une définition donnée. Drawdown : baisse relative depuis un sommet. Régression : ajustement d'une relation. R^2 : part de variation expliquée par l'ajustement dans son échantillon. Causalité temporelle du calcul : absence d'accès aux observations futures ; ce n'est pas la preuve d'une relation causale économique.

Sources primaires pour le contrat externe : https://www.coingecko.com/learn/download-bitcoin-historical-data ; https://support.coingecko.com/hc/en-us/articles/4538879945625-API-key-can-I-store-the-key-in-my-headers ; https://support.coingecko.com/hc/en-us/articles/23126953719321-Why-is-there-a-discrepancy-between-the-close-price-of-a-token-on-the-coins-id-market-chart-interval-daily-endpoint-and-the-OHLC-prices-for-the-same-token .

Le code livré, les tests, le README et pyproject.toml constituent la référence pour le comportement de cette version. Les annexes suivantes en donnent une copie de travail. Les numéros de ligne de cette copie sont indicatifs pour l'édition du guide et peuvent changer après de futurs commits.

Annexe A. Fiches des modules et lecture du code

Chaque source ci-dessous est reproduite depuis le dépôt local. Les numéros facilitent la discussion. Une ligne longue peut être poursuivie sur la ligne suivante sans nouveau numéro. Les commentaires et tests font partie du comportement documenté.

Méthode de lecture : identifier les entrées, suivre les transformations, repérer les effets de bord, puis associer une assertion de test à chaque garantie importante.

A1. pyproject.toml

Lire cette configuration comme un contrat d'exécution : identifier ce qui est actif, ce qui est exclu et ce qui dépend de l'environnement.

```
001  [build-system]
002  requires = ["setuptools>=68"]
003  build-backend = "setuptools.build_meta"
004
005  [project]
006  name = "crypto-etl-local"
007  version = "0.1.0"
008  requires-python = ">=3.10"
009  dependencies = ["pandas>=2.2,<3", "numpy>=1.26,<3", "requests>=2.32,<3", "streamlit>=1.40,<2",
     "plotly>=5.24,<7"]
010
011  [project.optional-dependencies]
012  dev = ["pytest>=8,<9", "ruff>=0.9,<1"]
013
014  [project.scripts]
015  crypto-etl = "crypto_etl.cli:main"
016
017  [tool.setuptools.packages.find]
018  where = ["src"]
019
020  [tool.pytest.ini_options]
021  testpaths = ["tests"]
022  pythonpath = ["src"]
023
024  [tool.ruff]
025  line-length = 100
026
```

A2. .env.example

Lire cette configuration comme un contrat d'exécution : identifier ce qui est actif, ce qui est exclu et ce qui dépend de l'environnement.

```
# Définir ces variables dans le terminal ; .env n'est pas chargé automatiquement.
COINGECKO_API_KEY=
CRYPTO_CURRENCY=usd
CRYPTO_DAYS=180
CRYPTO_DATA_DIR=data

```

A3. .gitignore

Lire cette configuration comme un contrat d'exécution : identifier ce qui est actif, ce qui est exclu et ce qui dépend de l'environnement.

```
.env
.venv/
__pycache__/
*.py[cod]
*.egg-info/
.pytest_cache/
.ruff_cache/
data/
*.db*
*.log
build/
dist/

```

A4. src/crypto_etl/config.py

Suivre Config, sa validation et load_config. Distinguer valeurs par défaut, variables réellement configurables et secret exclu de repr.

```
"""Configuration commune à la CLI et au dashboard."""

import os
from dataclasses import dataclass, field
from pathlib import Path


@dataclass(frozen=True)
class Config:
    api_key: str = field(default="", repr=False)
    currency: str = "usd"
    days: int = 180
    data_dir: Path = Path("data")
    coins: tuple[str, ...] = ("bitcoin", "ethereum", "solana")
    timeout: float = 30
    attempts: int = 4
    base_url: str = "https://api.coingecko.com/api/v3"

    def __post_init__(self):
        if not 1 <= self.days <= 365:
            raise ValueError("CRYPTO_DAYS doit être compris entre 1 et 365")
        if not self.currency.isalpha() or self.attempts < 1 or self.timeout <= 0:
            raise ValueError("Configuration invalide")

    def db_path(self, demo=False):
        return self.data_dir / ("demo.db" if demo else "market.db")


def load_config():
    return Config(
        api_key=os.getenv("COINGECKO_API_KEY", ""),
        currency=os.getenv("CRYPTO_CURRENCY", "usd").lower(),
        days=int(os.getenv("CRYPTO_DAYS", "180")),
        data_dir=Path(os.getenv("CRYPTO_DATA_DIR", "data")),
    )
```

A5. src/crypto_etl/client.py

Séparer décodage JSON et validation de structure. Compter les appels et les pauses. Repérer les erreurs immédiatement refusées.

```
"""Client HTTP : erreurs expurgées, tentatives bornées, aucune clé dans les logs."""

import time
from datetime import datetime, timezone
from email.utils import parsedate_to_datetime

import requests


def validate_payload(payload):
    if not isinstance(payload, dict):
        raise TypeError("Réponse JSON non objet")
    for key in ("prices", "market_caps", "total_volumes"):
        values = payload.get(key)
        if not isinstance(values, list):
            raise TypeError(f"Série absente ou invalide : {key}")
        if any(not isinstance(p, (list, tuple)) or len(p) != 2 for p in values):
            raise ValueError(f"Paires invalides : {key}")
    return payload


def retry_delay(value, attempt):
    try:
        return max(0, float(value))
    except (TypeError, ValueError):
        try:
            return max(
                0, (parsedate_to_datetime(value) - datetime.now(timezone.utc)).total_seconds()
            )
        except (TypeError, ValueError, OverflowError):
            return min(2**attempt, 30)


class Client:
    def __init__(self, config, session=None, sleep=time.sleep):
        self.config = config
        self.session = session or requests.Session()
        self.sleep = sleep


    def fetch(self, coin):
        c = self.config
        if not c.api_key:
            raise ValueError("COINGECKO_API_KEY requis pour une collecte réelle")
        for attempt in range(c.attempts):
            delay = min(2**attempt, 30)
            try:
                response = self.session.get(
                    f"{c.base_url}/coins/{coin}/market_chart",
                    params={"vs_currency": c.currency, "days": c.days, "interval": "daily"},
                    headers={"x-cg-demo-api-key": c.api_key},
                    timeout=c.timeout,
                )
            except requests.RequestException:
                pass
            else:
                if response.status_code == 200:
                    try:
```

```
                        return response.json()
                    except ValueError:
                        raise ValueError("JSON illisible") from None
                if response.status_code != 429 and response.status_code < 500:
                    raise ValueError(f"HTTP {response.status_code} : requête refusée")
                delay = retry_delay(response.headers.get("Retry-After"), attempt)
            if attempt + 1 < c.attempts:
                self.sleep(delay)
        raise RuntimeError("Échec API après le nombre maximal de tentatives")
```

A6. src/crypto_etl/transform.py

Suivre les clés d'alignement et le moment où la date devient quotidienne. Justifier chaque filtre et chaque keep='last'.

```
"""Alignement exact des timestamps puis dernier snapshot valide du jour UTC."""

import numpy as np
import pandas as pd

from .client import validate_payload
from .quality import quality_report


def transform(payload, coin, currency, days=180, now=None):
    validate_payload(payload)
    today = pd.Timestamp(now or pd.Timestamp.now(tz="UTC"))
    today = today.tz_localize("UTC") if today.tzinfo is None else today.tz_convert("UTC")
    today = today.normalize().tz_localize(None)
    start = today - pd.Timedelta(days=days)
    series = []
    duplicates = 0
    for key, name in (
        ("prices", "price"),
        ("market_caps", "market_cap"),
        ("total_volumes", "volume_24h"),
    ):
        part = pd.DataFrame(payload[key], columns=["source_timestamp", name])
        part["source_timestamp"] = pd.to_numeric(part["source_timestamp"], errors="coerce")
        part = part.dropna(subset=["source_timestamp"])
        duplicates += int(part.duplicated("source_timestamp").sum())
        part = part.drop_duplicates("source_timestamp", keep="last")
        series.append(part.set_index("source_timestamp"))
    frame = pd.concat(series, axis=1).reset_index()
    frame["date_utc"] = (
        pd.to_datetime(frame["source_timestamp"], unit="ms", utc=True, errors="coerce")
        .dt.tz_localize(None)
        .dt.normalize()
    )
    for column in ("price", "market_cap", "volume_24h"):
        frame[column] = pd.to_numeric(frame[column], errors="coerce")
        valid = np.isfinite(frame[column]) & (frame[column] >= 0)
        frame.loc[~valid, column] = np.nan

    invalid = frame["price"].isna() | (frame["price"] <= 0) | frame["date_utc"].isna()
    rejected = int(invalid.sum())
    excluded = int((frame["date_utc"] >= today).sum())
    frame = frame.loc[~invalid & frame["date_utc"].between(start, today, inclusive="left")]
    frame = frame.sort_values("source_timestamp").drop_duplicates("date_utc", keep="last")
    frame = frame.sort_values("date_utc").reset_index(drop=True)
    report = quality_report(
        frame, rejected, duplicates, excluded, start, today - pd.Timedelta(days=1)
    )
    frame["coin_id"] = coin
    frame["vs_currency"] = currency
    return frame, report
```

A7. src/crypto_etl/quality.py

Comparer calendrier attendu et dates acceptées. Expliquer pourquoi les compteurs ne s'additionnent pas forcément au nombre de points bruts.

```
"""Bilan sérialisable, sans interpolation."""

import pandas as pd


def quality_report(frame, rejected, duplicates, excluded, start, end):
    expected = pd.date_range(start, end, freq="D")
    observed = pd.DatetimeIndex(frame["date_utc"])
    prices = frame.set_index("date_utc")["price"].reindex(expected)
    returns = prices.pct_change(fill_method=None)
    return {
        "accepted": len(frame),
        "rejected": rejected,
        "duplicate_timestamps": duplicates,
        "excluded_current_or_future": excluded,
        "missing_days": expected.difference(observed).strftime("%Y-%m-%d").tolist(),
        "missing_market_cap": int(frame["market_cap"].isna().sum()),
        "missing_volume": int(frame["volume_24h"].isna().sum()),
        "large_changes": returns[returns.abs() > 0.5].index.strftime("%Y-%m-%d").tolist(),
    }
```

A8. src/crypto_etl/storage.py

Lire la clé primaire, les CHECK et l'upsert. Repérer les placeholders, le lot transactionnel et la fermeture explicite à améliorer.

```
"""Transactions SQLite : une crypto validée est chargée atomiquement."""

import sqlite3
from contextlib import contextmanager

import pandas as pd


@contextmanager
def connect(path):
    path.parent.mkdir(parents=True, exist_ok=True)
    with sqlite3.connect(path) as db:
        db.executescript("""
        CREATE TABLE IF NOT EXISTS daily_market_data (
            coin_id TEXT NOT NULL, vs_currency TEXT NOT NULL, date_utc TEXT NOT NULL,
            source_timestamp INTEGER NOT NULL, price REAL NOT NULL CHECK(price>0),
            market_cap REAL CHECK(market_cap>=0), volume_24h REAL CHECK(volume_24h>=0),
            collected_at TEXT NOT NULL,
            PRIMARY KEY (coin_id, vs_currency, date_utc));
        CREATE TABLE IF NOT EXISTS pipeline_runs (
            run_id TEXT PRIMARY KEY, started_at TEXT NOT NULL, ended_at TEXT,
            status TEXT NOT NULL, rows_processed INTEGER NOT NULL DEFAULT 0,
            details TEXT NOT NULL DEFAULT '{}');
        """)
        yield db


def upsert(db, frame, collected_at):
    rows = []
    for row in frame.to_dict("records"):
        rows.append(
            (
                row["coin_id"],
                row["vs_currency"],
                row["date_utc"].strftime("%Y-%m-%d"),
                int(row["source_timestamp"]),
                float(row["price"]),
                None if pd.isna(row["market_cap"]) else float(row["market_cap"]),

                None if pd.isna(row["volume_24h"]) else float(row["volume_24h"]),
                collected_at,
            )
        )
    with db:
        db.executemany(
            """
        INSERT INTO daily_market_data VALUES (?,?,?,?,?,?,?,?)
        ON CONFLICT(coin_id, vs_currency, date_utc) DO UPDATE SET
        source_timestamp=excluded.source_timestamp, price=excluded.price,
        market_cap=excluded.market_cap, volume_24h=excluded.volume_24h,
        collected_at=excluded.collected_at
        """,
            rows,
        )


def read_data(path, currency):
    with connect(path) as db:
```

```
        return pd.read_sql_query(
            "SELECT * FROM daily_market_data WHERE vs_currency=? ORDER BY date_utc",
            db,
            params=(currency,),
            parse_dates=["date_utc"],
        )


def latest_run(path):
    with connect(path) as db:
        db.row_factory = sqlite3.Row
        row = db.execute("SELECT * FROM pipeline_runs ORDER BY started_at DESC LIMIT
1").fetchone()
        return dict(row) if row else None
```

A9. src/crypto_etl/pipeline.py

Suivre le run_id du fichier brut à pipeline_runs. Montrer la limite de transaction par actif et le cas d'échec partiel.

```
"""Orchestration, archives brutes et suivi des échecs par crypto."""

import json
import logging
from datetime import datetime, timezone
from uuid import uuid4

from .client import Client
from .storage import connect, upsert
from .transform import transform

logger = logging.getLogger(__name__)


def utc_now():
    return datetime.now(timezone.utc).isoformat()


def run(config, fetch=None, demo=False):
    fetch = fetch or Client(config).fetch
    run_id, started = uuid4().hex, utc_now()
    raw_dir = config.data_dir / ("raw-demo" if demo else "raw") / run_id
    raw_dir.mkdir(parents=True, exist_ok=True)
    details, rows, successes = {}, 0, 0
    with connect(config.db_path(demo)) as db:
        with db:
            db.execute(
                "INSERT INTO pipeline_runs(run_id,started_at,status) VALUES (?,?,?)",
                (run_id, started, "running"),
            )
        for coin in config.coins:
            try:
                payload = fetch(coin)
                (raw_dir / f"{coin}.json").write_text(json.dumps(payload), encoding="utf-8")
                frame, report = transform(payload, coin, config.currency, config.days)
                if frame.empty:
                    raise ValueError("Aucune observation valide")
                upsert(db, frame, utc_now())

                details[coin] = {"status": "success", "quality": report}
                rows += len(frame)
                successes += 1
            except Exception as error:  # noqa: BLE001 — isoler chaque crypto dans le pipeline
                # Les exceptions externes peuvent contenir des secrets : ne pas les sérialiser.
                details[coin] = {"status": "failed", "error_type": type(error).__name__}
                logger.error("Collecte échouée pour %s (%s)", coin, type(error).__name__)
        status = (
            "success" if successes == len(config.coins) else "partial" if successes else
    "failed"
        )
        with db:
            db.execute(
                """UPDATE pipeline_runs SET ended_at=?, status=?, rows_processed=?,
                       details=? WHERE run_id=?""",
                (utc_now(), status, rows, json.dumps(details), run_id),
            )
    return {"run_id": run_id, "status": status, "rows": rows, "details": details}
```

A10. src/crypto_etl/indicators.py

Vérifier groupby, asfreq et ordre calcul/filtre. Expliquer pourquoi base 100 et drawdown dépendent de la sélection.

```
"""Fenêtres calendaires complètes ; volatilité non annualisée en pourcentage."""

import pandas as pd

from .trends import trend_indicators


def indicators(frame):
    parts = []
    for (coin, currency), group in frame.groupby(["coin_id", "vs_currency"]):
        daily = group.set_index("date_utc").sort_index().asfreq("D")
        daily["coin_id"], daily["vs_currency"] = coin, currency
        price = daily["price"]
        daily["return_1d"] = price.pct_change(fill_method=None) * 100
        daily["return_7d"] = (price / price.shift(7) - 1) * 100
        for window in (7, 30):
            daily[f"ma_{window}"] = price.rolling(window, min_periods=window).mean()
        daily["volatility_30d"] = daily["return_1d"].rolling(30, min_periods=30).std(ddof=1)
        daily = daily.join(trend_indicators(price))
        parts.append(daily.reset_index())
    return pd.concat(parts, ignore_index=True) if parts else frame.copy()


def select_period(frame, start, end):
    selected = frame.loc[frame["date_utc"].between(pd.Timestamp(start),
pd.Timestamp(end))].copy()
    if selected.empty:
        return selected
    groups = selected.groupby(["coin_id", "vs_currency"])["price"]
    selected["base_100"] = 100 * selected["price"] / groups.transform("first")
    selected["drawdown"] = (selected["price"] / groups.cummax() - 1) * 100
    return selected
```

A11. src/crypto_etl/trends.py

Suivre les fenêtres complètes, shift(1), les cas RSI constants et la logique du croisement ponctuel. Les seuils restent descriptifs.

```
"""Repères descriptifs causaux : aucun signal ne lit les prix futurs."""

import numpy as np
import pandas as pd


def slope_pct(values):
    x = np.arange(len(values), dtype=float)
    slope = np.polyfit(x, values, 1)[0]
    return 100 * slope / np.mean(values)


def fit_score(values):
    x = np.arange(len(values), dtype=float)
    fitted = np.polyval(np.polyfit(x, values, 1), x)
    total = np.sum((values - np.mean(values)) ** 2)
    return 0.0 if total == 0 else float(1 - np.sum((values - fitted) ** 2) / total)


def trend_indicators(price):
    result = pd.DataFrame(index=price.index)
    mean = price.rolling(20, min_periods=20).mean()
    std = price.rolling(20, min_periods=20).std(ddof=1)
    result["bollinger_lower"] = mean - 2 * std
    result["bollinger_upper"] = mean + 2 * std
    change = price.diff()
    gains = change.clip(lower=0).rolling(14, min_periods=14).mean()
    losses = (-change.clip(upper=0)).rolling(14, min_periods=14).mean()
    # RSI à moyennes simples, explicitement distinct du lissage de Wilder.
    result["rsi_14"] = 100 - 100 / (1 + gains / losses)
    result.loc[(losses == 0) & (gains > 0), "rsi_14"] = 100
    result.loc[(losses == 0) & (gains == 0), "rsi_14"] = 50
    result["trend_slope_30d"] = price.rolling(30, min_periods=30).apply(slope_pct, raw=True)
    result["trend_r2_30d"] = price.rolling(30, min_periods=30).apply(fit_score, raw=True)
    result["trend"] = "Indisponible"
    known = result.trend_slope_30d.notna()
    result.loc[known, "trend"] = "Sans tendance linéaire nette"
    coherent = known & (result.trend_r2_30d >= 0.5)

    result.loc[coherent & (result.trend_slope_30d > 0.1), "trend"] = "Haussière"
    result.loc[coherent & (result.trend_slope_30d < -0.1), "trend"] = "Baissière"
    result["support_20d"] = price.shift(1).rolling(20, min_periods=20).min()
    result["resistance_20d"] = price.shift(1).rolling(20, min_periods=20).max()
    result["breakout"] = "Indisponible"
    available = price.notna() & result.support_20d.notna()
    result.loc[available, "breakout"] = "Dans le canal précédent"
    result.loc[available & (price > result.resistance_20d), "breakout"] = "Nouveau sommet 20 j"
    result.loc[available & (price < result.support_20d), "breakout"] = "Nouveau creux 20 j"
    short, long = price.rolling(7).mean(), price.rolling(30).mean()
    difference = short - long
    result["ma_cross"] = "Indisponible"
    valid = difference.notna() & difference.shift(1).notna()
    result.loc[valid, "ma_cross"] = "Aucun croisement"
    result.loc[valid & (difference > 0) & (difference.shift(1) <= 0), "ma_cross"] = (
        "Croisement haussier"
    )
    result.loc[valid & (difference < 0) & (difference.shift(1) >= 0), "ma_cross"] = (
        "Croisement baissier"
```

```
058      )
059      result["momentum_30d"] = (price / price.shift(30) - 1) * 100
060      return result
```

A12. src/crypto_etl/demo.py

Identifier la formule synthétique et l'absence d'appel réseau. La date de fin change, même si les valeurs sont déterministes par position.

```
001  """Données déterministes synthétiques, sans requête HTTP."""
002
003  import math
004
005  import pandas as pd
006
007
008  def synthetic(coin, days=180):
009      base = {"bitcoin": 50000, "ethereum": 2500, "solana": 100}[coin]
010      dates = pd.date_range(
011          end=pd.Timestamp.now(tz="UTC").normalize() - pd.Timedelta(days=1), periods=days,
     freq="D"
012      )
013      payload = {"prices": [], "market_caps": [], "total_volumes": []}
014      for i, date in enumerate(dates):
015          timestamp = int(date.timestamp() * 1000)
016          price = base * (1 + i / 1000 + 0.08 * math.sin(i / 12 + base % 7))
017          for key, value in (
018              ("prices", price),
019              ("market_caps", price * 1e7),
020              ("total_volumes", price * 1e5 * (1 + 0.2 * math.cos(i))),
021          ):
022              payload[key].append([timestamp, value])
023      return payload
```

A13. src/crypto_etl/cli.py

Relier la commande à la base choisie et au code de sortie. L'absence de clé est vérifiée avant une collecte réelle.

```
"""Commandes manuelles utilisables également par un planificateur local."""

import argparse
import json
import logging

from .config import load_config
from .demo import synthetic
from .pipeline import run


def main():
    parser = argparse.ArgumentParser(description="ETL crypto local")
    parser.add_argument("command", choices=["update", "demo"])
    args = parser.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    config = load_config()
    if args.command == "update" and not config.api_key:
        parser.error("Définir COINGECKO_API_KEY ou utiliser la commande demo")
    result = run(
        config,
        fetch=(lambda coin: synthetic(coin, config.days)) if args.command == "demo" else None,
        demo=args.command == "demo",
    )
    print(json.dumps(result, indent=2, ensure_ascii=False))
    return 0 if result["status"] == "success" else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

A14. dashboard/app.py

Suivre les états vides, la devise, la sélection et les graphiques. Les écritures de marché appartiennent au pipeline, pas aux filtres.

```
"""Interface exclusivement locale : aucun appel API à l'interaction."""

import json

import pandas as pd
import plotly.express as px
import streamlit as st

from crypto_etl.config import load_config
from crypto_etl.indicators import indicators, select_period
from crypto_etl.storage import latest_run, read_data

st.set_page_config(page_title="Observatoire crypto", page_icon=" ", layout="wide")
st.title("Observatoire crypto")
config = load_config()
demo = st.sidebar.toggle("Démonstration synthétique", value=True)
if demo:
    st.warning("DÉMONSTRATION — données synthétiques, sans valeur de marché réelle.")
path = config.db_path(demo)
st.caption(f"Devise : {config.currency.upper()} · snapshots quotidiens UTC · base : {path}")
if not path.exists():
    st.info("Initialisez les données depuis le dossier du projet :")
    st.code("crypto-etl demo" if demo else "crypto-etl update")
    st.stop()
data = read_data(path, config.currency)
if data.empty:
    st.info("Base vide pour cette devise. Lancez crypto-etl demo ou crypto-etl update.")
    st.stop()
run = latest_run(path)
if run:
    st.caption(f"Dernière collecte : {run['ended_at']} · statut : {run['status']}")
    if run["status"] != "success":
        st.warning(
            "Mise à jour partielle ou échouée : les dernières données valides sont conservées."
        )
    reports = json.loads(run["details"])
    if any(
        v.get("quality", {}).get("missing_days")

        or v.get("quality", {}).get("missing_market_cap")
        or v.get("quality", {}).get("missing_volume")
        or v.get("quality", {}).get("rejected")
        for v in reports.values()
    ):
        st.warning("Données incomplètes ou rejetées : consultez le bilan qualité.")
    with st.expander("Bilan de la dernière collecte"):
        st.json(reports)
today = pd.Timestamp.now(tz="UTC").normalize().tz_localize(None)
latest = data.groupby("coin_id")["date_utc"].max()
if (latest < today - pd.Timedelta(days=1)).any():
    st.warning("Certaines données sont anciennes : relancez la collecte manuellement.")
coins = st.sidebar.multiselect("Cryptomonnaies", config.coins, default=list(config.coins))
dates = st.sidebar.date_input(
    "Période",
    value=(data.date_utc.min().date(), data.date_utc.max().date()),
    min_value=data.date_utc.min().date(),
    max_value=data.date_utc.max().date(),
)
```

```
show_ma = st.sidebar.checkbox("Moyennes mobiles", value=True)
if len(dates) != 2 or not coins:
    st.info("Sélectionnez des cryptomonnaies et une période complète.")
    st.stop()
calculated = indicators(data[data.coin_id.isin(coins)])
selected = select_period(calculated, *dates)
observed = selected.dropna(subset=["price"])
if observed.empty:
    st.info("Aucun prix disponible sur cette période.")
    st.stop()
if selected["price"].isna().any():
    st.warning("Des jours sont manquants ; les graphiques ne comblent pas les intervalles.")
if selected["volatility_30d"].isna().any():
    st.info("Certains indicateurs sont indisponibles : historique insuffisant ou jours manquants.")
overview, analysis, comparison = st.tabs(["Vue d’ensemble", "Analyse d’une crypto", "Comparaison"])
summary = observed.groupby("coin_id").tail(1).copy()
summary["performance_pct"] = summary["base_100"] - 100

columns = ["coin_id", "date_utc", "price", "return_1d", "return_7d", "performance_pct"]
with overview:
    st.dataframe(summary[columns], hide_index=True, use_container_width=True)
with analysis:
    coin = st.selectbox("Actif", coins)
    subset = selected[selected.coin_id == coin]
    last = subset.dropna(subset=["price"]).tail(1)
    if not last.empty:
        row = last.iloc[0]
        st.subheader(f"Tendance observée : {row['trend']}")
        st.caption(
            "Régression sur les 30 derniers jours : hausse/baisse si pente supérieure "
            "à ±0,1 % du prix moyen par jour et R² ≥ 0,5. Seuils descriptifs, "
            "sans validation prédictive. Un motif détecté ne prédit pas le prochain prix."
        )
        st.dataframe(
            last[
                [
                    "date_utc",
                    "trend_slope_30d",
                    "trend_r2_30d",
                    "rsi_14",
                    "momentum_30d",
                    "breakout",
                    "ma_cross",
                ]
            ],
            hide_index=True,
        )
    values = ["price", "ma_7", "ma_30"] if show_ma else ["price"]
    st.plotly_chart(
        px.line(subset, x="date_utc", y=values, title="Prix et moyennes mobiles"),
        use_container_width=True,
    )
    st.plotly_chart(
        px.line(
            subset,
            x="date_utc",

            y=["price", "bollinger_lower", "bollinger_upper", "support_20d", "resistance_20d"],
            title="Bandes de Bollinger et canal des 20 jours précédents",
        ),
        use_container_width=True,
    )
    rsi_chart = px.line(subset, x="date_utc", y="rsi_14", title="RSI 14 jours — moyennes simples")
```

```
    rsi_chart.add_hline(y=30, line_dash="dot")
    rsi_chart.add_hline(y=70, line_dash="dot")
    rsi_chart.update_yaxes(range=[0, 100])
    st.plotly_chart(rsi_chart, use_container_width=True)
    events = subset.loc[
        subset.breakout.isin(["Nouveau sommet 20 j", "Nouveau creux 20 j"])
        | subset.ma_cross.isin(["Croisement haussier", "Croisement baissier"])
    ]
    with st.expander("Motifs repérés : cassures du canal et croisements"):
        st.caption(
            "Le canal exclut le jour observé. Les événements utilisent seulement "
            "l'historique disponible à cette date. Les snapshots ne permettent pas "
            "de détecter des figures de chandeliers OHLC."
        )
        st.dataframe(events[["date_utc", "price", "breakout", "ma_cross"]], hide_index=True)
    st.plotly_chart(
        px.bar(subset, x="date_utc", y="volume_24h", title="Volume observé sur 24 h"),
        use_container_width=True,
    )
    for name in ("volatility_30d", "drawdown"):
        st.plotly_chart(
            px.line(subset, x="date_utc", y=name, title=f"{name} (%)"), use_container_width=True
        )
with comparison:
    st.plotly_chart(
        px.line(
            selected,
            x="date_utc",
            y="base_100",
            color="coin_id",
            title="Performance sur la sélection — base 100",

        ),
        use_container_width=True,
    )
    st.dataframe(
        summary[
            columns
            + ["volatility_30d", "drawdown", "trend", "rsi_14", "trend_r2_30d", "momentum_30d"]
        ],
        hide_index=True,
    )
st.download_button(
    "Exporter la sélection CSV",
    selected.to_csv(index=False).encode("utf-8"),
    file_name="crypto-demo.csv" if demo else "crypto-coingecko.csv",
    mime="text/csv",
)
st.caption(
    "Prix observés, sans clôture officielle. Volume 24 h non cumulable. "
    "Source des collectes réelles : [CoinGecko](https://www.coingecko.com/)."
)
```

A15. tests/test_core.py

Identifier les risques métier couverts et les assertions vérifiant un résultat, plutôt que seulement une absence d'exception.

```
"""Tests des risques métier, entièrement hors réseau."""

from unittest.mock import Mock

import pandas as pd
import pytest
import requests

from crypto_etl.client import Client, validate_payload
from crypto_etl.config import Config
from crypto_etl.indicators import indicators, select_period
from crypto_etl.pipeline import run
from crypto_etl.storage import connect, read_data, upsert
from crypto_etl.transform import transform


def payload(prices=(100, 105, 110), start="2024-01-01"):
    dates = pd.date_range(start, periods=len(prices), tz="UTC")
    pairs = [[int(d.timestamp() * 1000), p] for d, p in zip(dates, prices)]
    return {
        "prices": pairs,
        "market_caps": [[t, 10] for t, _ in pairs],
        "total_volumes": [[t, 20] for t, _ in pairs],
    }


@pytest.mark.parametrize("value", [None, [], {}, {"prices": [1]}])
def test_bad_payload(value):
    with pytest.raises((ValueError, TypeError)):
        validate_payload(value)


def test_empty():
    frame, quality = transform(payload(()), "bitcoin", "usd", 3, "2024-01-04")
    assert frame.empty and len(quality["missing_days"]) == 3


def test_alignment_duplicates_invalid_and_today():

    raw = payload((100, 105, -2, 120))
    raw["prices"] = raw["prices"][::-1] + [raw["prices"][0]]
    raw["market_caps"] = raw["market_caps"][1:]
    raw["total_volumes"][1][1] = float("inf")
    frame, quality = transform(raw, "bitcoin", "usd", 4, "2024-01-04")
    assert frame.price.tolist() == [100, 105]
    assert pd.isna(frame.iloc[0].market_cap)
    assert pd.isna(frame.iloc[1].volume_24h)
    assert quality["rejected"] == 1
    assert quality["duplicate_timestamps"] == 1
    assert quality["excluded_current_or_future"] == 1
    assert "2024-01-03" in quality["missing_days"]


def test_alignment_is_not_positional():
    raw = payload()
    raw["market_caps"] = [[raw["prices"][2][0], 30], [raw["prices"][0][0], 10]]
    frame, _ = transform(raw, "bitcoin", "usd", 3, "2024-01-04")
    assert frame.market_cap.iloc[0] == 10
```

```
    assert pd.isna(frame.market_cap.iloc[1])
    assert frame.market_cap.iloc[2] == 30


def test_daily_last_valid():
    raw = payload((100,))
    raw["prices"].append([raw["prices"][0][0] + 3600000, 110])
    frame, _ = transform(raw, "bitcoin", "usd", 1, "2024-01-02")
    assert frame.price.tolist() == [110]


def test_indicators_and_exact_calendar():
    frame, _ = transform(payload(tuple(range(100, 140))), "bitcoin", "usd", 40, "2024-02-10")
    result = indicators(frame)
    assert result.return_1d.iloc[1] == pytest.approx(1)
    assert result.ma_7.iloc[6] == 103
    assert result.return_7d.iloc[7] == pytest.approx(7)
    assert result.ma_30.iloc[29] == 114.5
    assert result.volatility_30d.iloc[30] ==

pytest.approx(result.return_1d.iloc[1:31].std(ddof=1))
    assert pd.isna(result.volatility_30d.iloc[29])
    selection = select_period(result, "2024-01-05", "2024-01-10")
    assert selection.base_100.iloc[0] == 100
    assert selection.ma_7.iloc[-1] == 106
    missing = indicators(frame.drop(index=3))
    assert pd.isna(missing.return_1d.iloc[4])
    assert pd.isna(missing.ma_7.iloc[6])
    assert pd.isna(missing.return_7d.iloc[10])


def test_drawdown():
    frame, _ = transform(payload((100, 120, 90)), "bitcoin", "usd", 3, "2024-01-04")
    result = select_period(indicators(frame), "2024-01-01", "2024-01-03")
    assert result.drawdown.iloc[-1] == -25
    assert result.base_100.iloc[-1] == 90


def test_upsert_and_revision(tmp_path):
    path = tmp_path / "test.db"
    frame, _ = transform(payload(), "bitcoin", "usd", 3, "2024-01-04")
    with connect(path) as db:
        upsert(db, frame, "first")
        upsert(db, frame, "second")
        frame.loc[0, "price"] = 200
        upsert(db, frame, "third")
    stored = read_data(path, "usd")
    assert len(stored) == 3 and stored.price.iloc[0] == 200


def test_client_retry_and_auth():
    response = Mock(status_code=429, headers={"Retry-After": "3"})
    good = Mock(status_code=200)
    good.json.return_value = payload()
    session, sleep = Mock(), Mock()
    session.get.side_effect = [response, good]
    client = Client(Config(api_key="secret"), session, sleep)
    assert client.fetch("bitcoin") == payload()

    sleep.assert_called_once_with(3)
    assert session.get.call_args.kwargs["headers"] == {"x-cg-demo-api-key": "secret"}
    session.get.side_effect = None
    session.get.return_value = response
    with pytest.raises(RuntimeError):
        client.fetch("bitcoin")
    assert session.get.call_count == 6
```

```
@pytest.mark.parametrize("status", [400, 401, 403])
def test_no_retry_for_bad_request(status):
    session = Mock()
    session.get.return_value = Mock(status_code=status)
    with pytest.raises(ValueError):
        Client(Config(api_key="secret"), session, Mock()).fetch("bitcoin")
    assert session.get.call_count == 1


def test_network_retry_and_invalid_json():
    session = Mock()
    session.get.side_effect = requests.ConnectionError("secret")
    with pytest.raises(RuntimeError, match="maximal"):
        Client(Config(api_key="secret", attempts=2), session, Mock()).fetch("bitcoin")
    assert session.get.call_count == 2
    session.get.side_effect = None
    session.get.return_value = Mock(status_code=200)
    session.get.return_value.json.side_effect = ValueError("secret")
    with pytest.raises(ValueError, match="JSON illisible"):
        Client(Config(api_key="secret"), session, Mock()).fetch("bitcoin")


def test_partial_pipeline_preserves_data(tmp_path):
    config = Config(data_dir=tmp_path)
    yesterday = (pd.Timestamp.now(tz="UTC").normalize() -
pd.Timedelta(days=1)).strftime("%Y-%m-%d")
    first = run(config, lambda _: payload((100,), yesterday))
    assert first["status"] == "success"


    def fetch(coin):
        if coin == "ethereum":
            raise RuntimeError("secret")
        return payload((200,), yesterday)

    second = run(config, fetch)
    assert second["status"] == "partial"
    data = read_data(config.db_path(), "usd").set_index("coin_id")
    assert data.loc["ethereum", "price"] == 100
    assert data.loc["bitcoin", "price"] == 200
    assert "secret" not in str(second)
    assert len(list((tmp_path / "raw").glob("*/*.json"))) == 5
```

A16. tests/test_dashboard.py

Comprendre AppTest, les données temporaires et les filtres. Ce test ne remplace pas un contrôle visuel dans le navigateur.

```
001  """Vérifier l'état vide et les filtres via le moteur Streamlit, sans serveur réseau."""
002
003  from pathlib import Path
004
005  from streamlit.testing.v1 import AppTest
006
007  from crypto_etl.config import Config
008  from crypto_etl.demo import synthetic
009  from crypto_etl.pipeline import run
010
011  APP = Path(__file__).resolve().parents[1] / "dashboard" / "app.py"
012
013
014  def test_dashboard_empty_and_demo(tmp_path, monkeypatch):
015      monkeypatch.setenv("CRYPTO_DATA_DIR", str(tmp_path))
016      app = AppTest.from_file(str(APP)).run()
017      assert not app.exception
018      assert "Initialisez" in app.info[0].value
019      config = Config(data_dir=tmp_path)
020      assert run(config, synthetic, demo=True)["status"] == "success"
021      app.run()
022      assert not app.exception
023      assert len(app.tabs) == 3
024      assert len(app.dataframe) == 4
025      app.sidebar.multiselect[0].set_value(["bitcoin"]).run()
026      assert not app.exception
027      assert len(app.dataframe[0].value) == 1
028      app.sidebar.multiselect[0].set_value([]).run()
029      assert not app.exception
030      assert "Sélectionnez" in app.info[0].value
```

A17. tests/test_trends.py

Relier les séries calculables à la main aux cas extrêmes ; lire le test de préfixe comme protection contre une fuite temporelle.

```
001  import numpy as np
002  import pandas as pd
003  import pytest
004
005  from crypto_etl.trends import trend_indicators
006
007
008  def test_linear_rising_and_falling():
009      rising = trend_indicators(pd.Series(np.arange(100.0, 160.0)))
010      last = rising.iloc[-1]
011      assert last.trend == "Haussière"
012      assert last.trend_r2_30d == pytest.approx(1)
013      assert last.trend_slope_30d == pytest.approx(100 / np.mean(np.arange(130.0, 160.0)))
014      assert last.rsi_14 == 100
015      assert last.breakout == "Nouveau sommet 20 j"
016      assert last.resistance_20d == 158
017      falling = trend_indicators(pd.Series(np.arange(160.0, 100.0, -1))).iloc[-1]
018      assert falling.trend == "Baissière" and falling.rsi_14 == 0
019      assert falling.breakout == "Nouveau creux 20 j"
020
021
022  def test_flat_and_bands():
023      result = trend_indicators(pd.Series([100.0] * 40))
024      assert result.iloc[-1].trend == "Sans tendance linéaire nette"
025      assert result.iloc[-1].rsi_14 == 50
026      assert result.iloc[-1].bollinger_lower == 100
027      assert result.iloc[-1].bollinger_upper == 100
028      assert pd.isna(result.rsi_14.iloc[13])
029
030
031  def test_missing_values_and_no_future_leakage():
032      price = pd.Series(np.arange(100.0, 160.0))
033      full = trend_indicators(price)
034      prefix = trend_indicators(price.iloc[:45])
035      pd.testing.assert_frame_equal(full.iloc[:45], prefix)
036      price.iloc[40] = np.nan
037      missing = trend_indicators(price)
038      assert pd.isna(missing.iloc[41].rsi_14)

039      assert missing.iloc[41].trend == "Indisponible"
040      assert missing.iloc[41].breakout == "Indisponible"
041
042
043  def test_crossing_is_an_event():
044      price = pd.Series([100.0] * 35 + list(np.arange(101.0, 131.0)))
045      result = trend_indicators(price)
046      assert result.ma_cross.iloc[35] == "Croisement haussier"
047      assert result.ma_cross.iloc[36] == "Aucun croisement"
```

A18. .github/workflows/ci.yml

Lire cette configuration comme un contrat d'exécution : identifier ce qui est actif, ce qui est exclu et ce qui dépend de l'environnement.

```
name: Qualité Python
on: [push, pull_request]
jobs:
  checks:
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest]
        python: ['3.10', '3.12']
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
          cache: pip
      - run: python -m pip install -e ".[dev]"
      - run: ruff check .
      - run: ruff format --check .
      - run: pytest
```

A19. .streamlit/config.toml

Lire cette configuration comme un contrat d'exécution : identifier ce qui est actif, ce qui est exclu et ce qui dépend de l'environnement.

```
001  [server]
002  # Surveiller aussi les modules du paquet installé en mode éditable.
003  folderWatchList = ["src"]
004  runOnSave = true
005
006  [browser]
007  gatherUsageStats = false
```

Annexe B. Commandes de révision

```
# Depuis le dossier du projet, PowerShell
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\python.exe -m ruff check .
.venv\Scripts\python.exe -m ruff format --check .
.venv\Scripts\python.exe -m crypto_etl.cli demo
.venv\Scripts\python.exe -m streamlit run dashboard/app.py

# Inspection Git
git status --short
git log --oneline
git remote -v
git ls-files

# Linux : remplacer les exécutables par .venv/bin/python
```

Ces commandes de vérification sont sans collecte réelle. La commande update nécessite une clé chargée dans l'environnement ; ne jamais la coller dans un document partagé ou un commit.